Mission:

The mission of the flight computer is to attack an enemy base while avoiding an obstacle, using GPS.

The Matlab Code

```
clc; close all; clear all;
format short
A = [-1.064 1.000 ; 290.26 0.00];
B = [-0.25 ; -331.40];
C = [-123.34 0.00 ; 0.00 1.00];
D = [-13.51 ; 0];
states = {'AoA' , 'q'};
inputs = {'\delta_c'};
outputs = {'Az', 'q'};
sys = ss(A,B,C,D,'statename',states,...
   'inputname',inputs,...
   'outputname',outputs);
%TF
TFs = tf(sys);
TF = TFs(2,1);
disp(pole(TF));
%LQR weight matrices
Q = [0.1 0 ; 0 0.1];
R = 0.5;
%LQR gain
[K,S,e] = lqr (A,B,Q,R);
fprintf('eigenvalues of  A-BK\n');
disp(eig(A-B*K));
fprintf('Feedback gain K');
disp(K)
%closed loop system
Acl = A-B*K;
Bcl = B;
syscl = ss(Acl,Bcl,C,D,'statename',states,...
   'inputname',inputs,...
   'outputname',outputs);
%our TF - closed loop
TF = tf(syscl);
TFc = TF(2,1);
%LQG Kalman filter design
G = eye(2);
H = 0*eye(2);
% Kalman Q,R noise matrices
Qbar = diag(0.00015*ones(1,2));
Rbar = diag(0.55*ones(1,2));
%define noisy system
sys_n = ss(A,[B G],C,[D H]);
```

```
[kest,L,P] = kalman(sys_n,Qbar,Rbar,0);
%kalman gain observer closed loop
Aob = A-L*C;
%display observer eigenvalues
fprintf('Observer eigenvalues/n');
disp(eig(Aob));
%%noise time constants
dT1 = 0.75;
dT2 = 0.25;
%missile/model parameters
R = 6371e3; %earth radius
Vel = 1021.08; %speed (m/s)
m2f = 3.2811; %meters to feet
%target location
LAT_TARGET = 34.6588;
LON_TARGET = -118.769745;
ELEV_TARGET = 795; %m - MSL
%initial location
LAT_INIT = 34.2329;
LON_INIT = -119.4573;
ELEV_INIT = 10000; %m p - MSL
%obstacle location
LAT_OBS = 34.61916;
LON_OBS = -118.8429;
d2r = pi/180; %degrees to radians
%convert to radian
l1 = LAT_INIT*d2r;
u1 = LON_INIT*d2r;
l2 = LAT_TARGET*d2r;
u2 = LON_TARGET*d2r;
dl = l2 - l1;
du = u2 - u1;
%haversine formula
a = sin(dl/2)^2 + cos(l1)*cos(l2)*sin(du/2)^2;
c = 2*atan2(sqrt(a),sqrt(1-a));
d = R*c; %horizontal distance (in m)
%initial range (pythagoras theorem)
r = sqrt(d^2+(ELEV_TARGET-ELEV_INIT)^2);
%initial azimuth
yaw_init = azimuth(LAT_INIT, LON_INIT, LAT_TARGET, LON_TARGET);
yaw = yaw_init*d2r;
%% initial flight path angle
dh = abs(ELEV_TARGET-ELEV_INIT);
FPA_INIT = atan(dh/d); %rad
```

Target Location

Latitude = 34.6588

Longitude = -118.769745
Elevation = 795

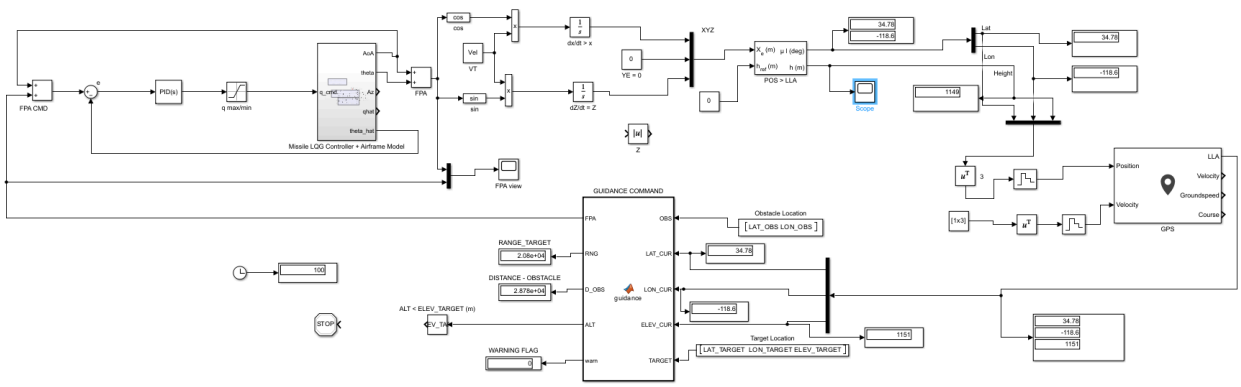
Initial Location

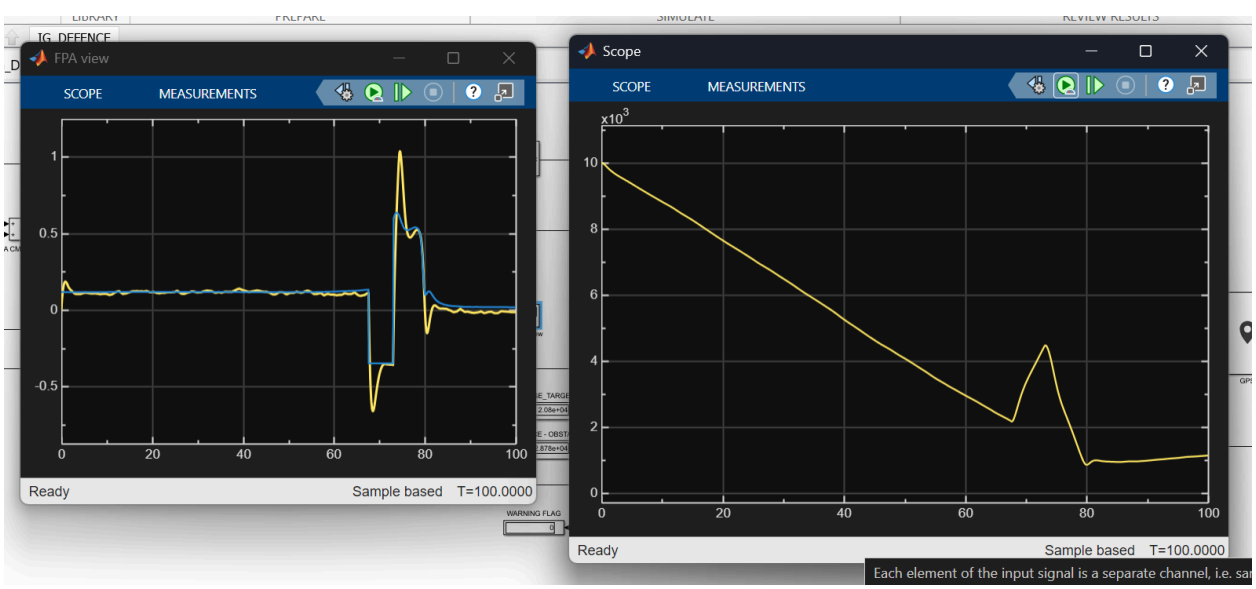
Latitude = 34.2329
Longitude = -119.4573
Elevation = 10000

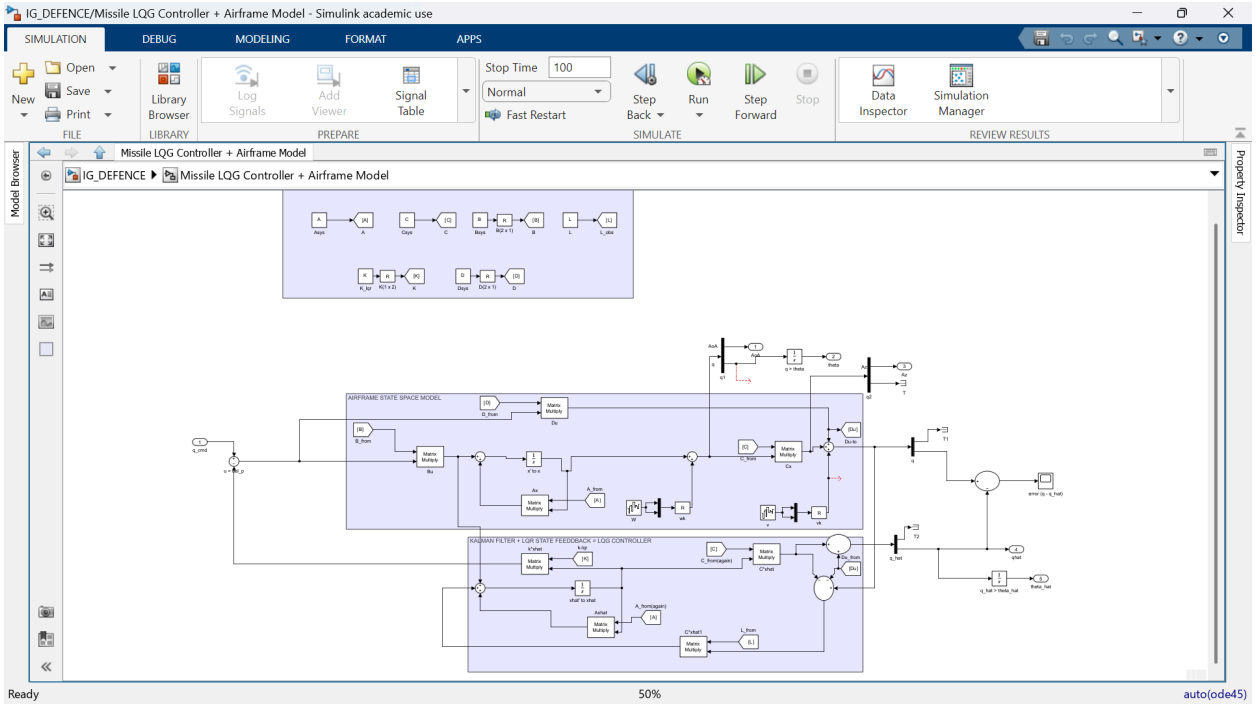
Obstacle Location

Latitude = 34.61916;
Longitude = -118.8429;

These are the graphs and the circuit images.






## Guidance Code

```
function [FPA, RNG, D_OBS, ALT, warn] = guidance(OBS, LAT_CUR, LON_CUR,
ELEV_CUR, TARGET)
R = 6371000;
d2r = pi/180;
% OBSTACLE LOCATION
LAT_OBS = OBS(1);
LON_OBS = OBS(2);
thres = 2500; % 2.5km Danger Zone
% TARGET LOCATION
LAT_TARGET = TARGET(1);
LON_TARGET = TARGET(2);
ELEV_TARGET = TARGET(3);
%% distance to target
l1 = LAT_CUR*d2r;
u1 = LON_CUR*d2r;
l2 = LAT_TARGET*d2r;
u2 = LON_TARGET*d2r;
% Current (10000) - Target (795) = +9205m.
dh = ELEV_CUR - ELEV_TARGET;
dl = l2 - l1;
du = u2 - u1;
a = sin(dl/2)^2 + cos(l1)*cos(l2)*sin(du/2)^2;
c = 2*atan2(sqrt(a),sqrt(1-a));
```

```
d = R*c;
%% distance from the obstacle
l3 = LAT_OBS*d2r;
u3 = LON_OBS*d2r;
dl_obs = l3 - l1;
du_obs = u3 - u1;
a_obs = sin(dl_obs/2)^2 + cos(l1)*cos(l3)*sin(du_obs/2)^2;
c_obs = 2*atan2(sqrt(a_obs),sqrt(1-a_obs));
d_obs = R*c_obs;
%% current range
range = sqrt(d^2+dh^2);
%% calculate commanded flight path setpoint
if d_obs >= thres
   w = 0;
   % Positive FPA commands the airframe to DIVE
   FPA_CMD = atan(dh/d);
else
   w = 1;
   % Negative FPA commands the airframe to CLIMB over the mountain
   FPA_CMD = -0.35;
end
%% output variables
FPA = FPA_CMD;
RNG = d;
D_OBS = d_obs;
warn = w;
ALT = ELEV_CUR;
```